OPERATING SYSTEMS LAB REPORT

Title: Orange Problem-2

Name: Mythri S
Course: B.Tech CSE (AI & ML)

SRN: PES1UG24AM419
University: PES University
Subject: Operating Systems

Objective

The objective of this assignment is to design and implement a simplified version control system similar to Git. The system supports object storage using hashing, tree-based directory representation, an index for staging files, and a commit mechanism to track changes over time.

Phase 1: Object Storage

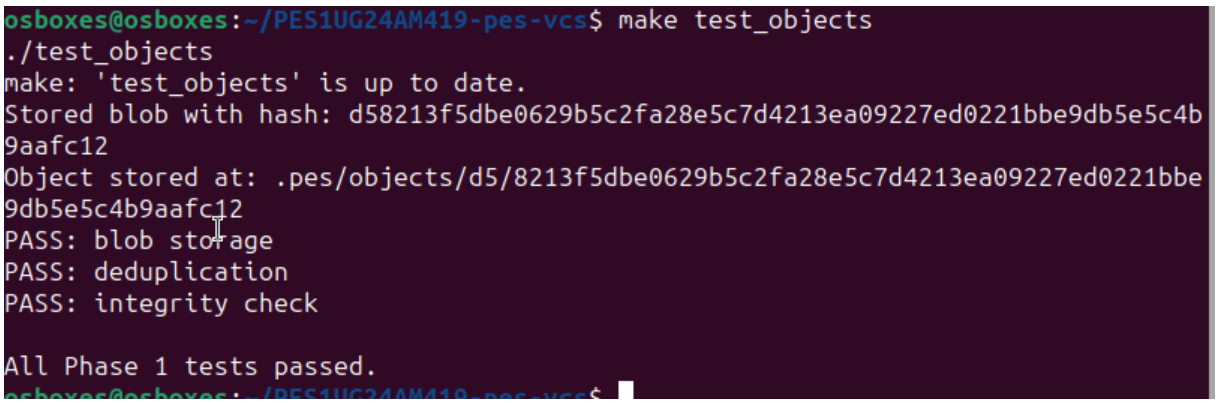
In this phase, blob objects were created to store file contents. Each file is hashed using the SHA-256 algorithm, and the content is stored in the `.pes/objects` directory.

Key Concepts:

- Content-addressable storage
- SHA-256 hashing
- Deduplication of objects

Screenshots:

SS1A:



```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

SS1B:

```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2: Tree Objects

Tree objects represent directory structures. Files and directories are serialized into a structured format and stored as tree objects.

Key Concepts:

- Directory hierarchy representation
- Serialization and parsing
- Deterministic structure generation

Screenshots:

SS2A:

```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

SS2B:

```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ find .pes/objects -type f
.pes/objects/d6/cdb071227e847b2160f72510a0ebe54c1f7fe960197716de844969bd288f82
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ xxd $(find .pes/objects -type f | head -1) | head -20
00000000: 7472 6565 2039 3500 3130 3036 3434 2068  tree 95.100644 h
00000010: 656c 6c6f 2e74 7874 0000 0102 0304 0506  ello.txt........
00000020: 0708 090a 0b0c 0d0e 0f10 1112 1314 1516  ................
00000030: 1718 191a 1b1c 1d1e 1f31 3030 3735 3520  .........100755
00000040: 7275 6e2e 7368 0001 0203 0405 0607 0809  run.sh..........
00000050: 0a0b 0c0d 0e0f 1011 1213 1415 1617 1819  ................
00000060: 1a1b 1c1d 1e1f 20                        ......
```

Phase 3: Index (Staging Area)

The index acts as a staging area where files are prepared before committing. It stores metadata such as file mode, hash, timestamp, and file path.

Key Concepts:

- File staging
- Metadata tracking
- Index file structure

Screenshots:

SS3A:

```
make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_obj
ects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
```

SS3B:

```
make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_obj
ects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
```

Phase 4: Commit System

Commits are created from the staged files. Each commit stores a tree reference, parent commit, author details, timestamp, and a commit message.

Key Concepts:

- Commit creation
- Parent-child relationship
- HEAD pointer management

Screenshots:

SS4A:

```
./pes log
[debug] commit_create start
[debug] tree_from_index ok
[debug] head_read found parent
[debug] commit struct filled
[debug] commit_serialize ok len=241
[debug] object_write commit ok
[debug] head_update ok
Committed: 0bbca36d8160... Add world
[debug] commit_create start
[debug] tree_from_index ok
[debug] head_read found parent
[debug] commit struct filled
[debug] commit_serialize ok len=244
[debug] object_write commit ok
[debug] head_update ok
Committed: 7d828229bacf... Add farewell
commit 7d828229bacfbbccfa16489042f820b41c87eda112bc85dbb6619f49c59ccbd7
Author: PES User <pes@localhost>
Date:   1776355115

    Add farewell

commit 0bbca36d8160c2d5f73ea588a598de5039d5d51eb9da9bf6ea7d690ca7d8fee0
```

SS4B:

```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0b/bca36d8160c2d5f73ea588a598de5039d5d51eb9da9bf6ea7d690ca7d8fee0
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7d/828229bacfbbccfa16489042f820b41c87eda112bc85dbb6619f49c59ccbd7
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/db/878763f912ae0d3fc3732b7e43868128bf4b4d2e103a58cdf3cf68ac2a7d87
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

SS4C:

```
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
7d828229bacfbbccfa16489042f820b41c87eda112bc85dbb6619f49c59ccbd7
ref: refs/heads/main
osboxes@osboxes:~/PES1UG24AM419-pes-vcs$
```

Explanation of Results

- Files are stored as objects using SHA-256 hashes in `.pes/objects`
- The index tracks staged files before committing
- Tree objects represent directory structure
- Commits maintain version history using parent links
- The HEAD pointer always refers to the latest commit

Q5: What is the role of hashing in version control systems?

ANS: Hashing ensures data integrity by generating a unique identifier for each file. It allows efficient storage, avoids duplication, and helps verify that file contents are not modified.

Q6: Difference between Git and this system

ANS:

- Git uses a Directed Acyclic Graph (DAG) for commits, while this system is linear

- Git supports branching and merging; this system does not

- Git is optimized for large-scale projects, while this is a simplified educational model